

FULL STACK DEVELOPMENT

FASE 2 | AULA 03 -

CI/CD COM GITHUB ACTION

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

EMSE

O QUE VEM POR AÍ?

Boas-vindas a uma aula empolgante pelo universo do Continuous Integration e Continuous Deployment (CI/CD), onde desvendaremos os mistérios por trás da eficiência no desenvolvimento de software, concentrando-nos especialmente nas poderosas ferramentas proporcionadas pelo GitHub Actions.

À medida que nos aprofundamos nesse tema, descobriremos como a automação inteligente pode revolucionar a forma como desenvolvemos e entregamos software. Prepare-se para uma experiência informativa e prática, na qual desmistificaremos os processos, compartilharemos dicas valiosas e capacitaremos você a incorporar eficientemente o CI/CD em seus projetos usando o GitHub Actions.

HANDS ON

Preparando-se para uma imersão prática e interativa, esta aula proporcionará uma jornada envolvente no universo do Docker e GitHub Actions. Durante o curso, você mergulhará nas funcionalidades do Docker, dominando seus principais comandos e aprendendo a criar arquivos Dockerfile.

Vamos explorar o poder do Docker Hub, descobrindo como compartilhar e distribuir suas imagens de contêiner de maneira eficiente. Para elevar ainda mais o nível, mostraremos como construir um pipeline com o GitHub Actions, permitindo a automação completa de todo esse processo.

SAIBA MAIS

O mundo do desenvolvimento de software está em constante evolução, impulsionado por avanços tecnológicos que buscam tornar o processo mais eficiente, confiável e ágil. Nesse cenário, surgiram duas práticas fundamentais que revolucionaram a forma como as equipes desenvolvem, testam e entregam software: Integração Contínua (CI) e Entrega Contínua (CD).

A Integração Contínua é um conceito que ganhou destaque no início dos anos 2000, em resposta aos desafios enfrentados por equipes de desenvolvimento ao lidar com a complexidade crescente dos projetos de software. Martin Fowler e Kent Beck foram pioneiros ao propor a prática de integrar código frequentemente, várias vezes ao dia.

A ideia central por trás da CI é simples, mas impactante: sempre que uma pessoa desenvolvedora faz uma alteração no código, essa mudança é automaticamente integrada ao repositório compartilhado. Isso visa evitar o acúmulo de alterações não testadas e facilitar a detecção precoce de conflitos ou erros.

A Entrega Contínua é uma extensão natural da Integração Contínua, elevando a eficiência do desenvolvimento para um novo patamar. Enquanto a CI se concentra na integração frequente, a CD busca automatizar todo o processo de entrega do software, desde a integração até a produção.

A automação desempenha um papel crucial na CD, permitindo que as equipes entreguem software funcional e testado em ambientes de produção a qualquer momento. Esse processo contínuo minimiza o tempo entre a conclusão do código e sua disponibilidade para os usuários finais, reduzindo os riscos associados a mudanças e aumentando a confiança na entrega.

A detecção rápida de problemas é, sem dúvida, um dos maiores benefícios da CI. A automação da integração do código ao repositório compartilhado permite que a equipe identifique imediatamente conflitos entre as alterações propostas, evitando o acúmulo de incompatibilidades que poderiam resultar em conflitos complexos e demorados.

Além disso, os testes automatizados realizados durante cada integração desempenham um papel crucial na identificação de possíveis problemas. Se um teste

falha, a equipe é notificada de imediato, indicando que a alteração introduziu um problema. Essa abordagem proativa previne a propagação de erros para fases posteriores do desenvolvimento, permitindo que sejam corrigidos antes de se tornarem problemas mais sérios.

A melhoria na qualidade do código é uma consequência natural desse processo. A detecção rápida de problemas no âmbito da CI incentiva os desenvolvedores e desenvolvedoras a escreverem código mais robusto desde o início. O conhecimento de que suas alterações serão integradas e testadas automaticamente cria um estímulo adicional para manter padrões elevados de qualidade, contribuindo para um código mais confiável e de fácil manutenção.

A CI também facilita a integração contínua de novos recursos. Ao identificar e corrigir problemas prontamente, a equipe pode avançar com confiança para a implementação de novas funcionalidades, sabendo que a base existente está estável e pronta para a introdução de melhorias.

Com isso surge o Github Actions, uma ferramenta poderosa para automação de fluxos de trabalho no desenvolvimento de software, proporcionando uma abordagem flexível e integrada para a execução de ações automatizadas diretamente no GitHub. Ao compreender a natureza do GitHub Actions, é possível otimizar significativamente os processos de desenvolvimento, desde a integração contínua até a entrega contínua.

GitHub Actions, em sua essência, é uma plataforma de automação que permite a criação de workflows personalizados, acionados por eventos específicos no ciclo de vida do desenvolvimento. Esses eventos podem variar desde a criação de um pull request até a realização de um push em uma determinada branch. Dessa forma, cada ação é desencadeada em resposta a um evento específico, proporcionando um ambiente altamente personalizável e adaptável às necessidades do projeto.

A verdadeira força do GitHub Actions reside na sua integração nativa com o ecossistema do GitHub. Ao criar um arquivo de configuração YAML chamado `.github/workflows/main.yml` no repositório, as pessoas desenvolvedoras podem definir workflows que automatizam tarefas cruciais. Esses workflows consistem em uma série de jobs, cada um contendo um conjunto de passos específicos que serão executados sequencialmente.

Por exemplo, consideremos um cenário comum de CI/CD. Quando uma pessoa desenvolvedora realiza um push em uma branch, o GitHub Actions pode automaticamente executar uma série de ações, como a compilação do código, execução de testes e, se bem-sucedido, o deploy do software em um ambiente de produção. Isso cria uma linha de automação contínua, garantindo que cada mudança seja validada e implementada de maneira consistente.

O GitHub Actions não se limita apenas à automação de tarefas no próprio GitHub. Sua flexibilidade permite a integração com diversas ferramentas e serviços externos, ampliando seu escopo de aplicação. Desde notificações em plataformas de mensagens até a construção e envio de artefatos para serviços de hospedagem, o GitHub Actions oferece uma solução centralizada para a automação de diversas atividades.

Adicionalmente, a biblioteca de ações do GitHub fornece um vasto conjunto de ações predefinidas que podem ser incorporadas diretamente nos workflows. Essas ações cobrem uma ampla gama de tarefas, desde a interação com serviços de nuvem até a execução de comandos específicos no ambiente de execução do workflow.

Em conclusão, o GitHub Actions é mais do que uma ferramenta de automação; é uma peça essencial no toolkit do(a) desenvolvedor(a) moderno(a). Ao proporcionar uma maneira intuitiva e integrada de automatizar fluxos de trabalho, o GitHub Actions eleva a eficiência do desenvolvimento de software, permitindo que as equipes foquem no que realmente importa: criar software de qualidade, de forma ágil e consistente. Incorporar o GitHub Actions no processo de desenvolvimento é mais do que uma escolha técnica, é uma decisão estratégica para impulsionar a excelência no ciclo de vida do software.

E como estamos falando sobre CI/CD, não podemos deixar de falar do Docker Hub, uma plataforma central que desempenha um papel crucial na distribuição, compartilhamento e gerenciamento de imagens de contêineres. Este repositório global oferece uma infraestrutura robusta para armazenamento e acesso a imagens Docker, proporcionando uma série de vantagens para profissionais em desenvolvimento e equipes de operações.

O Docker Hub é mais do que apenas um depósito para imagens; é uma plataforma de colaboração, permitindo que desenvolvedores(as) compartilhem suas

criações com a comunidade global. Este aspecto colaborativo é uma das principais vantagens, visto que as pessoas desenvolvedoras podem explorar, reutilizar e contribuir para um vasto conjunto de imagens já disponíveis no Docker Hub. Isso acelera significativamente o desenvolvimento, permitindo que equipes aproveitem soluções existentes, evitando a necessidade de reinventar a roda a cada projeto.

A centralização oferecida pelo Docker Hub simplifica consideravelmente o processo de gerenciamento de imagens. Equipes podem organizar suas imagens em repositórios, cada um contendo várias versões e tags. Isso não apenas facilita a organização, mas também simplifica a referência e a implementação de imagens específicas em diferentes ambientes. A capacidade de versionamento e tag torna a identificação e o rastreamento de alterações uma tarefa eficiente.

Outro benefício essencial do Docker Hub é a integração direta com ferramentas e serviços populares. Muitas plataformas de orquestração de contêineres, como Kubernetes, suportam nativamente o Docker Hub, simplificando a implementação e a escalabilidade de aplicações em contêineres. Além disso, a integração com sistemas de CI/CD é direta, permitindo a construção, teste e implantação contínua com facilidade.

A segurança é uma consideração fundamental em qualquer ambiente de desenvolvimento e implementação. O Docker Hub oferece recursos de segurança que ajudam a proteger imagens contra ameaças comuns. Isso inclui varreduras automáticas de vulnerabilidades nas imagens, garantindo que apenas imagens seguras e atualizadas sejam utilizadas em ambientes de produção.

No contexto empresarial, o Docker Hub também oferece opções avançadas de gerenciamento de acesso, controle de políticas e suporte a ambientes privados. Isso é crucial para organizações que necessitam de um controle mais rigoroso sobre suas imagens, especialmente ao lidar com dados sensíveis ou regulamentados.

O Docker Hub não é apenas um repositório de imagens, é um ecossistema dinâmico que impulsiona a colaboração, simplifica o gerenciamento e promove a eficiência no desenvolvimento de contêineres. Ao aproveitar as vantagens oferecidas pelo Docker Hub, as equipes podem acelerar o ciclo de vida de desenvolvimento, promover a reutilização de recursos e garantir a segurança e confiabilidade em suas aplicações baseadas em contêineres. Incorporar o Docker Hub como parte integrante

da estratégia de contêineres é mais do que uma escolha técnica; é uma decisão estratégica para impulsionar a excelência no mundo dinâmico da computação em contêineres.

EMEND

O QUE VOCÊ VIU NESTA AULA?

Nesta aula nós mergulhamos no mundo da Dockerização de projetos, uma prática essencial para garantir a portabilidade e consistência em ambientes de desenvolvimento e produção. Aprendemos não apenas a containerizar nosso projeto, mas também a levá-lo para o Docker Hub, um repositório central que se tornou fundamental no ecossistema de contêineres. A jornada não parou por aí; avançamos para a criação de um workflow no GitHub Actions, sincronizado com a branch principal. Esse processo de integração contínua e entrega contínua (CI/CD) oferece uma solução eficaz para automatizar testes e implementações, proporcionando eficiência e confiabilidade ao nosso ciclo de desenvolvimento. Ao fechar esta seção do conteúdo, consolidamos não apenas as habilidades técnicas práticas, mas também compreendemos a importância estratégica dessas práticas modernas no desenvolvimento de software.

REFERÊNCIAS

DOCKER. **Overview of Docker Hub.** 2023. Disponível em: <
<https://docs.docker.com/docker-hub/>>. Acesso em: 04 abr. 2024.

DOCKER. **Overview of Docker Build.** 2023. Disponível em: <
<https://docs.docker.com/build/>>. Acesso em: 04 abr.. 2024.

GITHUB. **Entendendo o GitHub Actions.** 2023. Disponível em: <
<https://docs.github.com/pt/actions/learn-github-actions/understanding-github-actions>
>. Acesso em: 04 abr.. 2024.

PALAVRAS-CHAVE

Palavras-chave: Docker. Pipeline. CI/CD. GITHUB. GITHUB ACTIONS.

EMSE

POS TECH